

AWS DevOps Practice Roadmap

AWS DevOps Practice Roadmap — 15 Projects

Built around what you've already done: two-tier VPC/ALB/EC2 with Terraform, a cost kill-switch (Lambda/SNS/Budgets/QuickSight), CloudWise (ECS Fargate + Timestream + Bedrock FinOps), STAYHAVEN (MERN + GitHub Actions CI/CD), VeloRent (Docker + 8-stage Jenkins), and a Kinesis/Glue/Athena data pipeline. You're past "hello world" — this roadmap skips repeat ground and pushes into orchestration, native AWS CI/CD, Kubernetes, multi-account governance, DR, and chaos/observability, which are the gaps most DevOps engineers get tested on in interviews and real jobs.

Each project lists: **new skills**, **AWS services**, and a **done-when** bar so you know when to move on.

Phase 1 — Solidify core CI/CD + IaC patterns (Projects 1-3)

1. Immutable infrastructure with Packer + Auto Scaling

Bake a golden AMI (app pre-installed) with Packer, deploy it via an Auto Scaling Group behind an ALB, driven entirely by Terraform. Trigger a rolling AMI update and watch ASG cycle instances with zero downtime. - **New skills:** immutable infra pattern, Packer, ASG lifecycle hooks, launch templates - **Services:** EC2, Packer, Auto Scaling, ALB, Terraform, CloudWatch - **Done when:** you can ship a new AMI version and the ASG replaces all instances with no failed health checks.

2. Native AWS CI/CD pipeline (no Jenkins/GitHub Actions)

Rebuild STAYHAVEN's or VeloRent's pipeline using 100% AWS-native tooling: CodeCommit (or GitHub source), CodeBuild for build/test, CodeDeploy for deployment (in-place or blue/green), CodePipeline to orchestrate. - **New skills:** AWS-native CI/CD (distinct from Jenkins/Actions you already know), buildspec.yml, appspec.yml - **Services:** CodePipeline, CodeBuild, CodeDeploy, CodeCommit/CodeStar Connections, S3 (artifacts), IAM - **Done when:** a git push auto-deploys to EC2 or ECS with automatic rollback on failed health checks.

3. Multi-environment IaC with remote state + workspaces

Take your two-tier Terraform project and split it into reusable modules (network, compute, security) with separate dev/staging/prod environments using Terraform workspaces or a directory-per-env pattern, remote state in S3 + DynamoDB locking. - **New skills:** module design, remote state locking, environment promotion - **Services:** S3, DynamoDB, Terraform, IAM - **Done when:** you can terraform apply the same module against 3 environments with only variable files changing.

Phase 2 — Containers, orchestration, and serverless depth (Projects 4-9)

4. EKS cluster from scratch

Deploy a Kubernetes cluster on EKS with Terraform (eksctl or Terraform EKS module), deploy a containerized app with a Helm chart, set up Horizontal Pod Autoscaler and an Ingress via ALB Controller. - **New skills:** Kubernetes on AWS (new — your prior container work is ECS/Fargate + Docker), Helm, HPA - **Services:** EKS, ECR, ALB Ingress Controller, IAM Roles for Service Accounts (IRSA) - **Done when:** a load test triggers HPA to scale pods, and Ingress routes traffic correctly.

5. Blue/Green + Canary deployments on ECS Fargate

Extend CloudWise's ECS Fargate stack: implement blue/green deployment via CodeDeploy for ECS, then a canary rollout (shift 10% → 50% → 100% traffic) using ALB weighted target groups. - **New skills:** deployment strategies (blue/green vs canary), traffic shifting - **Services:** ECS Fargate, CodeDeploy, ALB, CloudWatch Alarms (auto-rollback trigger) - **Done when:** a bad deployment auto-rolls-back based on a CloudWatch alarm mid-canary.

6. Event-driven serverless pipeline

Build a fully serverless data/event pipeline: S3 upload triggers Lambda → processes → writes to DynamoDB → publishes to

EventBridge → fans out to SQS/SNS subscribers, orchestrated with Step Functions for multi-step workflows with retries. - **New skills:** event-driven architecture, Step Functions state machines, DLQs - **Services:** Lambda, S3, DynamoDB, EventBridge, SQS, SNS, Step Functions - **Done when:** you can inject a failure and watch Step Functions retry then route to a dead-letter queue.

7. API Gateway + Lambda microservice with auth

A serverless REST/HTTP API (API Gateway + Lambda + DynamoDB) with Cognito authentication, request throttling, and custom domain via Route 53 + ACM. - **New skills:** serverless API patterns, Cognito auth, custom domains for APIs - **Services:** API Gateway, Lambda, DynamoDB, Cognito, Route 53, ACM, CloudFront (optional) - **Done when:** unauthenticated requests are rejected, authenticated ones work end-to-end, and it's on a custom domain.

8. Container security + supply chain hardening

Take any existing containerized project (VeloRent is a good fit) and add: ECR image scanning, Checkov/tfsec in the pipeline (you've used Checkov for CloudWise — apply it here as a pipeline gate, not just a one-off scan), and least-privilege IAM per task/pod. - **New skills:** shift-left security, CI/CD security gates, IAM least-privilege for containers - **Services:** ECR (image scanning), CodeBuild, IAM, Secrets Manager - **Done when:** a pipeline run fails the build on a critical CVE or a Checkov policy violation.

9. Centralized secrets + config management

Refactor 2-3 of your existing projects to pull secrets from Secrets Manager / SSM Parameter Store instead of hardcoded env vars, with automatic rotation for at least one secret (e.g., a DB password). - **New skills:** secret rotation, parameter hierarchies, runtime secret injection - **Services:** Secrets Manager, Systems Manager Parameter Store, Lambda (rotation function), RDS - **Done when:** rotating the secret doesn't break the running app (zero-downtime rotation).

Phase 3 — Observability, governance, and resilience at scale (Projects 10-15)

10. Full observability stack

Add distributed tracing and dashboards to CloudWise or any multi-service project: X-Ray for request tracing across Lambda/ECS, custom CloudWatch metrics + dashboards, and since you already scoped Prometheus/Grafana for CloudWise — actually stand it up this time on ECS or EKS. - **New skills:** distributed tracing, the Grafana/Prometheus setup you deprioritized earlier, alerting thresholds - **Services:** X-Ray, CloudWatch (Logs Insights, Dashboards, Alarms), Managed Prometheus/Grafana or self-hosted - **Done when:** you can trace a single request across 3+ services and see it in a dashboard.

11. Multi-account landing zone with AWS Organizations

Set up a mini Control Tower-style landing zone: separate accounts for dev/prod/security/log-archive using AWS Organizations, Service Control Policies (SCPs), centralized CloudTrail + Config aggregation. (CloudWise already uses Organizations for multi-account support — go deeper here on governance, not just billing.) - **New skills:** account vending, SCPs, centralized logging/compliance - **Services:** AWS Organizations, Control Tower, CloudTrail (org trail), AWS Config, IAM Identity Center - **Done when:** an SCP successfully blocks a disallowed action (e.g., launching an instance type) in a member account.

12. Security posture automation

Enable GuardDuty, Security Hub, and AWS Config rules across your accounts; auto-remediate one finding (e.g., an open security group) with an EventBridge rule → Lambda remediation function. - **New skills:** automated security remediation, compliance-as-code - **Services:** GuardDuty, Security Hub, AWS Config, EventBridge, Lambda - **Done when:** you manually open a security group to 0.0.0.0/0 and it's auto-remediated within minutes.

13. Disaster recovery — multi-region active/passive

Take your two-tier architecture and make it DR-ready: RDS cross-region read replica (or DynamoDB Global Tables), S3 cross-region replication, Route 53 health-check-based failover. - **New skills:** RTO/RPO thinking, cross-region replication, failover automation - **Services:** Route 53 (failover routing), RDS/DynamoDB Global Tables, S3 CRR, CloudWatch - **Done when:** you kill the primary region's endpoint and Route 53 fails over automatically within your target RTO.

14. Chaos engineering + auto-healing

Use AWS Fault Injection Simulator (FIS) to inject failures (kill an EC2 instance, throttle an API, spike CPU) into one of your existing stacks and verify ASG/ECS/CloudWatch alarms detect and self-heal without manual intervention. - **New skills:**

chaos engineering, failure-mode validation - **Services:** AWS FIS, Auto Scaling, ECS, CloudWatch, SNS (alerting) - **Done when:** you can run a FIS experiment and the system recovers with zero manual steps.

15. Cost + FinOps capstone (extend CloudWise)

Push CloudWise further: add Compute Savings Plans/Reserved Instance recommendation logic, budget-based auto-scaling limits, and a scheduled Lambda that tags/reports untagged resources for cost allocation — tying together everything from projects 1-14 into one governed, observable, cost-aware platform. - **New skills:** FinOps automation at scale, resource tagging governance - **Services:** Cost Explorer API, Budgets, Compute Optimizer, Lambda, Timestream, QuickSight/Bedrock (already in CloudWise) - **Done when:** the platform can flag an untagged/oversized resource and recommend a specific right-sizing action automatically.

AWS DevOps Service Reference (organized by category)

Compute & Containers EC2, Auto Scaling, ECS, ECS Fargate, EKS, ECR, Lambda, Elastic Beanstalk, App Runner, Batch

CI/CD CodePipeline, CodeBuild, CodeDeploy, CodeCommit / CodeStar Connections, CodeArtifact

Infrastructure as Code CloudFormation, Terraform (AWS provider), CDK, Packer

Networking VPC, ALB/NLB, Route 53, CloudFront, API Gateway, Transit Gateway, PrivateLink, WAF, Shield

Monitoring & Observability CloudWatch (Metrics, Logs, Alarms, Dashboards, Logs Insights), X-Ray, CloudTrail, Managed Prometheus/Grafana

Security & Governance IAM (+ IAM Identity Center), Secrets Manager, Systems Manager (Parameter Store, Session Manager), GuardDuty, Security Hub, AWS Config, Organizations, Control Tower, KMS, Checkov/tfsec (third-party, but pipeline-critical)

Messaging & Orchestration SQS, SNS, EventBridge, Step Functions

Storage & Data S3, EBS, EFS, DynamoDB, RDS, Aurora, Timestream, Kinesis, Glue, Athena

Cost / FinOps Cost Explorer, AWS Budgets, Compute Optimizer, Trusted Advisor, Savings Plans

Resilience AWS Fault Injection Simulator (FIS), AWS Backup, Route 53 health checks/failover

Third-party tools you'll pair with these (already in your stack) Docker, Jenkins, GitHub Actions, Helm, Prometheus/Grafana, Checkov

Suggested pacing

At ~4 hrs/day, budget roughly: - Phase 1 (Projects 1-3): ~1.5-2 weeks - Phase 2 (Projects 4-9): ~4-5 weeks - Phase 3 (Projects 10-15): ~5-6 weeks

Total: ~10-13 weeks for all 15, assuming you're running this alongside your existing backend/DSA tracks, not instead of them.